EE3600 – Practical 2:
DC Motor Speed Control with P/PI/PID

Optional extension: gearbox-output position control

0. Purpose

Continue from your completed Practical 1 speed curve and measured first-order motor model. **Speed control and its Pico implementation are required; position control is optional.** Choose **one applicable tuning method** from the lecture, calculate initial gains, test them in simulation, implement them, and assess the measured response. PI is sufficient if it meets the requirements; using all three PID terms is not a requirement.

The three routes are analytical design (Section 3), Ziegler–Nichols where applicable (Section 4), and computational tuning (Section 5). Manual adjustment may refine any route. You do not have to perform all three methods on the motor.

1. Review and Simplification

A) Carry forward your Practical 1 result

The full voltage-to-speed model is

$$\frac{\omega(s)}{V_a(s)} = \frac{K_t}{LJs^2 + (RJ + LD)s + (RD + K_tK_b)}.$$

Neglecting the fast electrical dynamics gives the first-order form already fitted in Practical 1. Reuse your measured and validated model:

$$G_{P1}(s) = \frac{\omega_{\text{out}}(s)}{V_a(s)} = \frac{\overline{K}}{\tau s + 1}.$$

Record $\overline{K}$ in output $\text{rad}\,\text{s}^{-1}\,\text{V}^{-1}$, τ in seconds, and the motor, gearbox and load used. Attach the P1 speed curve and model-validation plot. **A new identification experiment is not required.**

B) Convert once into the controller's units

The supplied controller uses motor-shaft RPM and signed fractional duty u (for example, $u = 0.40$ means 40%). Let N be motor revolutions per output revolution and V_s the actual motor-board supply voltage. Using the nominal approximation $V_a \approx V_s u$,

$$G_n(s) = \frac{n_{\text{motor}}(s)}{u(s)} = \frac{K_{\text{PWM}}}{\tau s + 1}, \qquad \boxed{K_{\text{PWM}} = \overline{K}\, N\, \frac{60}{2\pi}\, V_s}.$$

K_{PWM} has units motor RPM per fractional duty; τ is unchanged. The website's **Convert P1 model** and **Use converted model** buttons perform this handover. If your P1 result is already in these units, select the already-converted option. Driver losses and supply variation can cause prediction errors; assess these in the control trials.

C) Optional position model

Position is the integral of speed. For the website's gearbox-output angle in degrees,

$$G_\theta(s) = \frac{\theta_{\text{out}}(s)}{u(s)} = \frac{A}{s(\tau s + 1)}, \qquad A = \frac{6K_{\text{PWM}}}{N} = \frac{180}{\pi} V_s \overline{K}.$$

Use this conversion for output degrees; do not multiply by the gear ratio again.

2. Baseline Closed-Loop System and Performance

A) Establish a proportional-control baseline

Use unity negative feedback: $e = r - y$, controller $u = C(s)e$, and plant output $y = G(s)u$. The reference-to-output transfer function is

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}.$$

For speed with $C(s) = K_p$,

$$T_n(s) = \frac{K_{\mathrm{PWM}}K_p}{\tau s + 1 + K_{\mathrm{PWM}}K_p}, \qquad \tau_{\mathrm{cl}} = \frac{\tau}{1 + K_{\mathrm{PWM}}K_p}, \qquad \frac{e_{ss}}{r_f} = \frac{1}{1 + K_{\mathrm{PWM}}K_p}$$

for a nonzero step from rest. Increasing K_p speeds up the ideal response and reduces its steady-state error, but cannot make that error zero at finite gain. Choose a feasible reference from your P1 speed curve and a modest gain that respects the duty limit. Simulate this baseline and record the error as well as the response time.

B) Required speed-control targets

Use the same reference step, load, supply and duty limit when comparing designs.

Quantity	Teaching target and definition
Overshoot	$\leq$ 5% of the commanded step magnitude, measured beyond the commanded final value.
Rise time t_r	≤ 0.50 s; time between 10% and 90% of the commanded change.
Settling time t_s	≤ 1.00 s; enter the target band and remain there for the rest of the assessed run, including at least a 2 s final dwell.
Final tracking error	Mean absolute error over the last 2 s $\leq \max(0.02\lvert r_f\rvert, 2q_n)$ motor RPM. Also report signed bias.

For C_m encoder counts per motor revolution and the 50 ms speed window,

$$q_n = \frac{60}{0.050\, C_m}\ \text{RPM}, \qquad b_n = \max(0.02|r_f - r_0|, 2q_n)\ \text{RPM}.$$

b_n is the speed settling band about r_f. The website reports the actual band. With $C_m = 44$, two quanta are about 54.5 motor RPM; state when this resolution floor governs your result. Missing or insufficient data cannot establish a pass. These are lab design targets to be checked on the real kit.

3. Analytical Design

A) Required speed task: a straightforward initial PI design

Choose a desired closed-loop time constant T_c. To obtain $T_n(s) = 1/(T_c s + 1)$ in the ideal model, choose

$$C(s) = K_p + \frac{K_i}{s} = \frac{\tau s + 1}{K_{\mathrm{PWM}}T_c s}, \qquad \boxed{K_p = \frac{\tau}{K_{\mathrm{PWM}}T_c}, \quad K_i = \frac{1}{K_{\mathrm{PWM}}T_c}, \quad K_d = 0.}$$

This controller places its zero at the model's pole. For the resulting first-order response, $t_r = \ln(9)T_c$, the 2% settling time is $t_s = \ln(50)T_c$, nominal overshoot is zero, and the stable ideal loop has zero step error. Therefore,

$$T_c \leq \min\left(\frac{0.50}{\ln 9}, \frac{1.00}{\ln 50}\right) = 0.228\ \text{s}.$$

For example, $T_c = 0.20$ s predicts $t_r = 0.439$ s and $t_s = 0.782$ s. Insert *your* K_{PWM} and τ to calculate the gains. This is an initial design, not a measured result: model error, finite duty and measurement delay may require slower tuning or a smaller step. Check the complete response in simulation before implementation.

3B) Optional position design: P, then PD if needed

For the position plant, P control gives

$$T_\theta(s) = \frac{AK_p}{\tau s^2 + s + AK_p}, \qquad \omega_n^2 = \frac{AK_p}{\tau}, \qquad 2\zeta\omega_n = \frac{1}{\tau}.$$

Here $\zeta\omega_n$ is fixed by the motor: **P control cannot choose damping and natural frequency independently.** Test whether a suitable K_p meets the targets; do not solve for two independent performance choices using only one gain. The plant integrator already gives zero nominal step-position error in a stable ideal loop.

The lecture's optional position *simulation* exercise uses overshoot $\leq 5\%$, $0.25 \leq t_s \leq 0.80$ s and zero nominal steady-state error. Select a damping ratio and settling-time aim, then estimate

$$\zeta \geq 0.70 \quad \text{for a 5\% overshoot limit,} \qquad \omega_n \approx \frac{4}{\zeta t_s}.$$

For the hardware's derivative-on-measurement PD form, $u = K_p(r-\theta) - K_d\dot{\theta}$ in the ideal unfiltered case,

$$T_\theta(s) = \frac{AK_p}{\tau s^2 + (1 + AK_d)s + AK_p}, \qquad \boxed{K_p = \frac{\tau\omega_n^2}{A}, \quad K_d = \frac{2\zeta\omega_n\tau - 1}{A}, \quad K_i = 0.}$$

Use nonnegative gains and validate the chosen response. If the formula gives $K_d < 0$, revisit the target or test P control; do not enter a negative derivative gain. The standard overshoot/settling formulas guide the ideal design. Filtering, sampling and limits change the implemented response. A conventional PD acting on error also adds a reference-path zero, so its step response must be checked separately.

4. Empirical Design: Ziegler–Nichols

A) Check whether the method applies

The ultimate-gain method requires a finite positive proportional gain K_u at which the loop has sustained oscillation with period T_u. **Neither ideal model in this practical has such a gain.** For $K_p > 0$, the speed characteristic polynomial is $\tau s + 1 + K_{\mathrm{PWM}}K_p$; the position polynomial is $\tau s^2 + s + AK_p$. Both remain stable for all positive K_p in these ideal models.

Thus, do not keep increasing gain until the ideal model appears to oscillate, and do not force the physical motor into an instability test. If the instructor supplies an appropriate higher-order/delayed model or a suitable recorded experiment, identify K_u, T_u from that example. Otherwise, explain why this route is inapplicable and choose analytical or computational tuning.

B) Convert an applicable K_u, T_u into parallel gains

Controller	K_p	K_i	K_d
P	$0.5K_u$	0	0
PI	$0.45K_u$	$1.2K_p/T_u$	0
PID	$0.6K_u$	$2K_p/T_u$	$K_pT_u/8$

These are initial values; Ziegler–Nichols can be too aggressive for a 5% overshoot target. Simulate and refine them. K_u here means *ultimate controller gain*, not the motor-model gain K_{PWM}.

The lecture's reaction-curve alternative also needs an applicable S-shaped response and a resolved positive delay L. A delay-free first-order model has $L = 0$, and the position plant has an integrator, so its usual table cannot be applied directly here. For a model $Ke^{-Ls}/(Ts+1)$, the reaction-curve PID rule includes plant gain: $K_p = 1.2T/(KL)$; omitting K assumes unit gain.

5. Computational Tuning and Manual Refinement

A) Build and tune the complete feedback model

Use MATLAB/Simulink or another suitable control-design tool with the Section 1 model and units. Include an error summing junction with negative feedback.

1. Select PI for the speed task initially. Enter your measured K_{PWM} and τ, chosen reference step and duty limits. For optional position use $A/[s(\tau s + 1)]$ and start with P or PD.
2. Use a PID tuning tool or the lecture's parameter-search approach. State the search range and evaluate overshoot, rise time, settling and tracking error against Section 2.
3. Use **parallel** gains. For $K_d > 0$, match the Pico: PI on error minus filtered D on measured output, with $T_f = 0.020$ s (derivative setpoint weight zero). If software reports $K_p(1+1/(T_i s)+ T_d s)$, convert $K_i = K_p/T_i$ and $K_d = K_p T_d$. For PI, set $K_d = 0$.
4. Validate the complete reference-to-output response and applied duty. Include saturation and realistic measurement dynamics when available. The firmware controls every 10 ms and estimates speed over 50 ms. Do not add the same measurement lag twice if it was already included in your P1 fitted model.

MATLAB's `stepinfo` can help measure timing. Check its final-value and threshold settings: assess the response against the *commanded* target and step, including any tracking offset. Use the website's documented metrics for the hardware CSV.

B) Refine the selected design by observation

Change one setting at a time under the same test conditions. If overshoot, saturation or noise grows, reduce the demand or slow the tuning. Integral action removes nominal speed offset but may worsen overshoot when output is limited. Add derivative only if it helps; the firmware filters it and applies it to the measured output.

Keep the initial and final gains and explain adjustments. If the initial design meets the targets, retain it and repeat the run to check consistency. A software suggestion still requires measured validation.

6. Comparison and Discussion

Summarise your chosen route using the following table; separate simulations from measurements. Include K_p, K_i, K_d and their units in an adjacent table or caption.

Case	Overshoot	t_r (s)	t_s (s)	Final error / bias
P baseline: simulation				
Initial design: simulation				
Initial hardware trial				
Final hardware trial / repeat				

Overlay reference and measured speed for the initial and final trials. Show applied duty and any saturation. Discuss:

- Which targets were met, and which were not? State the settling/error band used.
- How did gain changes affect the response and duty? What explains model/hardware differences?
- Why was the tuning route applicable? For Ziegler–Nichols, was a finite K_u available?

For optional position, distinguish the lecture's simulation target from the browser's hardware guide: $t_r \leq 0.75$ s and $t_s \leq 1.50$ s. Report numerical results and both contexts clearly.

Course reference: `EE3600-PID Controller.pdf`, including the three tuning methods and their applicability. This task sheet defines the required speed task and optional position extension.

7. Raspberry Pi Pico Implementation

A) Required speed-control trial

Use the supplied Practical 2 firmware and the website's `practical2-control.html` page. The Pico runs the controller; the browser sets gains, plots measurements and saves results. No Thonny control loop is needed.

1. If needed, install `EE3600_Practical_2_Pico.uf2` on the original RP2040 Pico: with motor power off, hold BOOTSEL while connecting USB and copy the UF2 to `RPI-RP2`. Keep the established Motor 1 and encoder wiring.
2. Open the page in desktop Chrome or Edge; choose **Connect Pico**. Close other applications using that serial port. In **Hardware setup**, enter the actual supply, verified encoder counts/revolution and gear ratio, and agreed duty, speed and trial limits. Choose **Apply configuration** while disarmed.
3. Secure the motor with the shaft free to turn. Use **Optional diagnostics** for a brief low-duty direction check: positive duty must give positive speed. Reuse the established supply/current limits; the page does not measure voltage or current. Resolve sign errors before closed-loop operation.
4. Enter the P1 model; choose **Convert P1 model**, then **Use converted model**. Calculate gains by your chosen route. **Calculate suggestions** only displays values; enter your chosen gains into **Speed control** yourself.
5. Enter an achievable **Target motor speed (RPM)** and gains in the units below, leaving correction headroom. Keep derivative filter $T_f = 0.020$ s initially. Use a finite trial with at least 2 s after settling; the default 10 s accommodates the stated targets.
6. Choose **Arm & run speed trial**. Use **STOP** if needed; confirm DISARMED and allow for coasting. Afterwards, choose **Analyse response** and **Save CSV**. Save each run before the next. Stop before changing gains/configuration, then repeat under the same conditions.

Mode	K_p	K_i	K_d
Speed	duty / motor RPM	duty / (motor RPM $\cdot$ s)	duty $\cdot$ s / motor RPM
Position	duty / output degree	duty / (output degree $\cdot$ s)	duty $\cdot$ s / output degree

Gains use fractional duty; traces and CSV duty columns display percent. Do not copy gains from a percent-duty/output-RPM simulator without conversion. The firmware includes output limiting, anti-windup and filtered derivative on measurement. Settings must be reapplied after reboot.

B) Optional position implementation

With the motor stopped, choose **Zero position**. Enter an output angle within the travel limit and your position gains; choose **Arm & run position trial**. The angle is relative to this zero, in output degrees, without shortest-path wrapping. Save the response and duty CSV. Use error band $b_\theta = \max(2^\circ, 720/(C_m N)\,^\circ)$ and at least 2 s final dwell. Encoder-derived position cannot independently measure gearbox backlash or slip.

8. Deliverables

- P1 model and validation evidence, unit conversion to K_{PWM}, τ, and confirmed configuration.
- Tuning method, applicability, calculations/search settings and baseline/design simulations.
- Initial/final gains with units; measured reference/speed/duty CSVs and plots; the Section 6 comparison against targets and discussion of adjustments and model/hardware differences.
- Position design/results only if completing the optional extension.